TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



SOFTWARE ARCHITECTURE

Bài tập lớn:

SwiftRide

VER 1.0

Môn học: Kiến trúc Phần mềm
Học kỳ: I – Năm học 2026–2027

TP. HỒ CHÍ MINH, NĂM 2026

Mục lục

1 Tóm tắt **2**

2 Bài toán: SwiftRide **2**
2.1 Bối cảnh . . . 2
2.2 Kịch bản đầy đủ: từ lúc đặt xe đến lúc thanh toán . . . 2
2.3 Công thức tính giá . . . 3
2.4 Quyết định thiết kế: 3 microservice cho SwiftRide . . . 3

3 Kiến trúc tổng quan và Client Architecture **3**
3.1 Sơ đồ kiến trúc logic . . . 3
3.2 API Gateway . . . 4
3.3 Ba cơ sở dữ liệu khác nhau . . . 4

4 Flowchart: vòng đời một chuyến đi **4**

5 Chi tiết ba microservice **6**
5.1 Trip Service . . . 6
5.2 Matching và Pricing Service . . . 6
5.3 Payment Service . . . 6

6 Áp dụng nguyên tắc SOLID (kèm ví dụ .NET) **6**
6.1 S – Single Responsibility Principle . . . 6
6.2 O – Open/Closed Principle . . . 6
6.3 L – Liskov Substitution Principle . . . 7
6.4 I – Interface Segregation Principle . . . 7
6.5 D – Dependency Inversion Principle . . . 7

7 Các design pattern sử dụng trong SwiftRide **8**

8 Ngăn xếp công nghệ .NET **9**
8.1 Cấu trúc thư mục đề xuất . . . 9

9 Trade-off và yêu cầu chất lượng **10**
9.1 Kiến trúc mã nguồn . . . 10
9.2 Nhất quán và độ tin cậy . . . 10
9.3 Bảo mật và khả năng quan sát . . . 10

1 Tóm tắt

Tài liệu này xây dựng một **dự án mẫu** tên là **SwiftRide**. Dự án áp dụng kiến trúc **Microservices** với **tối đa 3 dịch vụ** và **3 cơ sở dữ liệu khác nhau**, trong đó *dịch vụ Thanh toán (Payment)* là bắt buộc.
Tài liệu trình bày:
(1) **Client Architecture**;
(2) sơ đồ **flowchart** vòng đời chuyến đi;
(3) cách áp dụng **nguyên tắc SOLID**;
(4) các**design pattern** phổ biến;
(5) ngăn xếp công nghệ **.NET**;
(6) các trade-off quan trọng khi lựa chọn microservices.

Hệ thống SwiftRide có rất nhiều dịch vụ (Driver Discovery, Route, Time Estimation, Surge Pricing, Tax, Toll, Matching…). Tuy nhiên, trong bài tập lớn này chúng ta sẽ có **3 dịch vụ cốt lõi**. Chúng ta có thể chạy một luồng đơn giản; message broker, Outbox và observability là phần mở rộng.

2 Bài toán: SwiftRide

2.1 Bối cảnh

Hệ thống đặt xe là một **sàn giao dịch hai phía** (two-sided marketplace) kết nối *rider* (người đi) với *driver* (tài xế). Giá trị cốt lõi là *độ tin cậy*: cân bằng cung – cầu theo thời gian và không gian.

2.2 Kịch bản đầy đủ: từ lúc đặt xe đến lúc thanh toán

Để sinh viên dễ hình dung, dưới đây là **hành trình end-to-end** của một chuyến đi, kèm dịch vụ nào chịu trách nhiệm ở mỗi bước:

1. **Đặt xe.** Rider mở app, chọn điểm đón và điểm đến, nhấn "Đặt xe". Yêu cầu đi qua *API Gateway* tới **Trip Service**.
2. **Tạo chuyến.** Trip Service tạo một chuyến đi mới ở trạng thái `Requested` và lưu vào *SQL Server*.
3. **Tìm tài xế và báo giá.** Trip Service hỏi **Matching và Pricing Service**: dịch vụ này tìm tài xế gần nhất (truy vấn vị trí trên *MongoDB*) và tính giá theo công thức bên dưới.
4. **Rider xác nhận giá.** Giá được hiển thị cho rider. Nếu rider *từ chối*, chuyến bị hủy. Nếu *đồng ý*, chuyển sang bước kế.
5. **Đề xuất cho tài xế.** Tài xế nhận lời mời. Nếu tài xế *từ chối* và chưa quá k lần thử $\Rightarrow$ quay lại tìm tài xế khác; nếu vượt quá k lần $\Rightarrow$ chuyến *không thành công*.
6. **Thực hiện chuyến.** Tài xế nhận chuyến: *đang đến* $\rightarrow$ *đã đón khách* $\rightarrow$ *đã trả khách*. Mỗi mốc, tài xế cập nhật và Trip Service đổi trạng thái tương ứng.
7. **Thanh toán.** Khi trả khách xong, Trip Service phát sự kiện để **Payment Service** trừ tiền rider, ghi sổ cái vào *PostgreSQL*.
8. **Kết thúc.** Khi thanh toán hoàn tất, chuyến đi đóng lại (trạng thái cuối).

2.3 Công thức tính giá

Giá tiền một chuyến đi được ước lượng theo công thức:

$$p \;=\; \big((c_d \cdot d + c_t \cdot t) \cdot s + c_m\big) \cdot \eta \cdot (1 + T)$$

trong đó d là quãng đường, t là thời gian ước lượng, s là hệ số giá động (surge), c_m là phí phụ (toll), η là hệ số khuyến mãi, T là thuế.

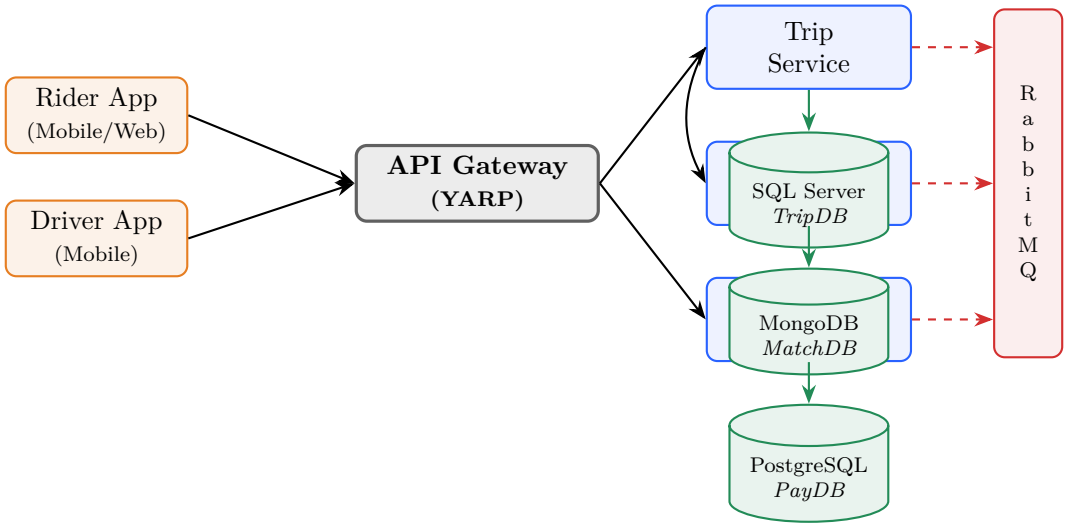
2.4 Quyết định thiết kế: 3 microservice cho SwiftRide

1. **Trip Service** – quản lý *vòng đời chuyến đi* (state machine).
2. **Matching và Pricing Service** – tìm tài xế gần nhất và *tính giá*.
3. **Payment Service** (**bắt buộc**) – xử lý *thanh toán* và sổ cái giao dịch.

3 Kiến trúc tổng quan và Client Architecture

3.1 Sơ đồ kiến trúc logic

Client (ứng dụng Rider và Driver) không gọi trực tiếp từng microservice mà đi qua **API Gateway**. Các dịch vụ giao tiếp đồng bộ qua REST/gRPC khi cần phản hồi ngay và *bất đồng bộ* qua message broker (RabbitMQ) để phát sự kiện. Sơ đồ dưới đây là *logical/container diagram*.



Các tầng trong Client Architecture:

- **Presentation (Client):** Rider App / Driver App xử lý UI/UX, kiểm tra dữ liệu đầu vào và gọi API; quy tắc nghiệp vụ có tính quyết định vẫn phải được kiểm tra ở server.
- **Edge / Gateway:** API Gateway đảm nhận định tuyến, xác thực (JWT), rate-limit, và *aggregation* (gom dữ liệu cho màn hình).
- **Application (Microservices):** mỗi dịch vụ sở hữu dữ liệu riêng (*database-per-service*) – không chia sẻ database.
- **Integration:** RabbitMQ truyền sự kiện (`TripAccepted`, `RideCompleted`, `PaymentSettled`…).

3.2 API Gateway

Dùng YARP. YARP (*Yet Another Reverse Proxy*) do Microsoft phát triển, tích hợp với ASP.NET Core và cấu hình đơn giản, lựa chọn phù hợp cho dự án .NET. Ocelot dễ tiếp cận trong .NET; gateway "phổ biến nhất" trong mọi bối cảnh. Ocelot dễ tiếp cận trong .NET;
Phân biệt: API Gateway là điểm vào và định tuyến chung. BFF (*Backend for Frontend*) cung cấp API riêng cho từng loại client. SwiftRide dùng một API Gateway; có thể tách Rider BFF và Driver BFF khi hai giao diện có nhu cầu khác biệt.

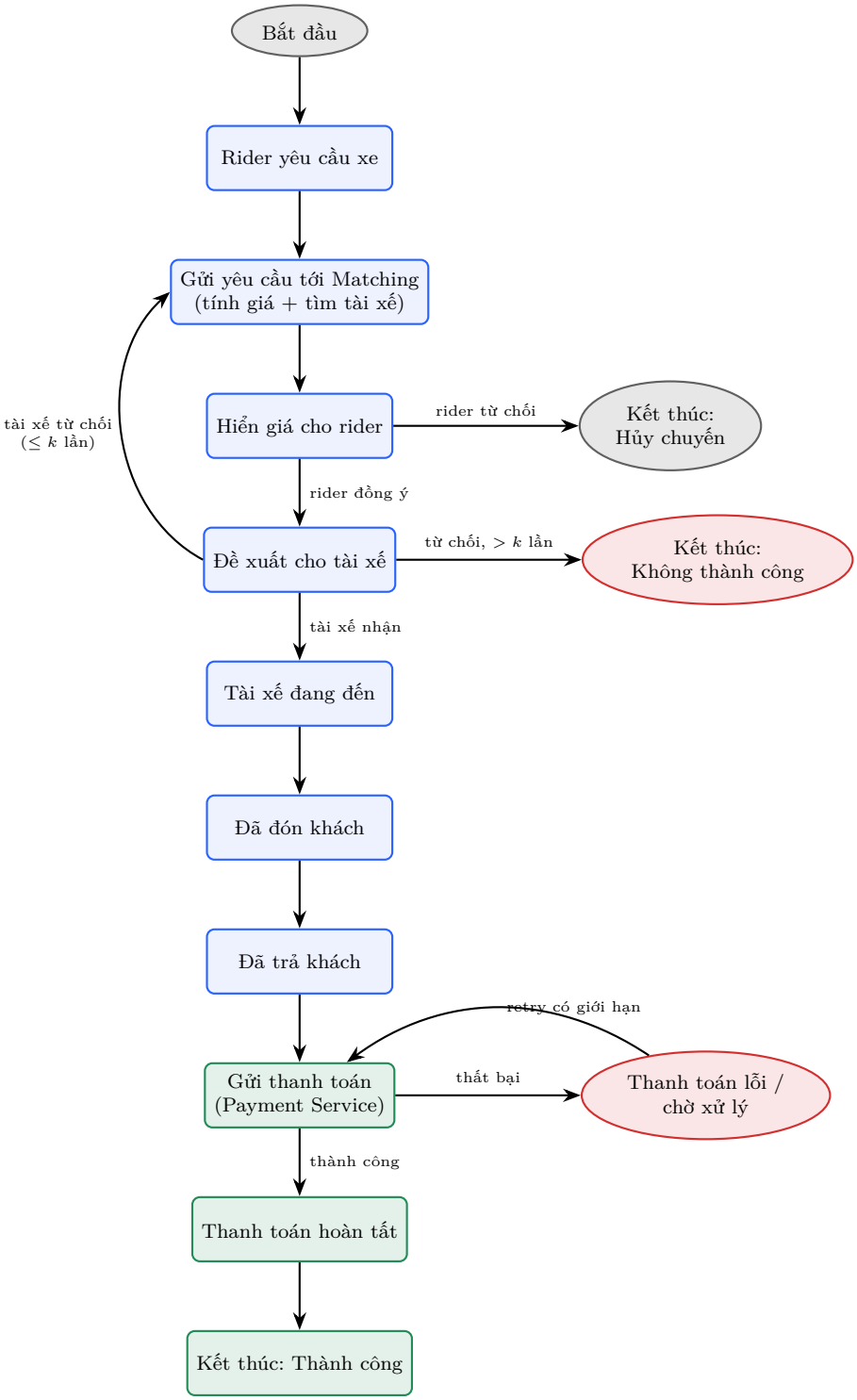
3.3 Ba cơ sở dữ liệu khác nhau

Dịch vụ	CSDL	Lý do lựa chọn
Trip Service	SQL Server (quan hệ)	Trạng thái chuyến đi cần *ACID*, ràng buộc toàn vẹn và chuyển trạng thái nhất quán; mô hình quan hệ rõ ràng.
Matching và Pricing	MongoDB (document)	Dữ liệu vị trí tài xế / đặc trưng (route, rider, driver) có cấu trúc linh hoạt, ghi/đọc nhanh, hỗ trợ *geo-spatial index*.
Payment Service	PostgreSQL (quan hệ)	Sổ cái giao dịch kiểu *double-entry*, cần transaction mạnh, kiểm toán, và đảm bảo nhất quán tiền bạc.

Polyglot Persistence: "mỗi dịch vụ chọn CSDL phù hợp nhất với nghiệp vụ của nó" thay vì ép một CSDL duy nhất cho toàn hệ thống. Tuy nhiên, ba công nghệ CSDL cũng làm tăng chi phí vận hành, sao lưu và giám sát. Trong Bài tập lớn này, chấp nhận sinh viên dùng cùng một loại CSDL nhưng tách schema/database theo service được xem là hợp lệ.

4 Flowchart: vòng đời một chuyến đi

Sơ đồ máy trạng thái dưới đây mô hình hóa một chuyến đi trong SwiftRide. Trip Service là nơi *giữ trạng thái*; các dịch vụ khác phản ứng qua sự kiện. Mỗi mũi tên mang *một nhãn điều kiện duy nhất*.



Đọc sơ đồ: chuyến đi bắt đầu khi rider yêu cầu xe → gửi sang Matching để tính giá + tìm tài xế → rider xác nhận giá → đề xuất cho tài xế. Nếu tài xế từ chối quá k lần thì *không thành công*; nếu chưa quá k lần thì *quay lại* tìm tài xế khác. Nếu chấp nhận: đón khách → trả khách → **Payment Service** xử lý thanh toán. Nếu lỗi tạm thời, hệ thống retry có giới hạn với cùng một *idempotency key*; nếu vẫn lỗi, chuyến ở trạng thái chờ xử lý thay vì trừ tiền lặp hoặc báo thành công sai.

5 Chi tiết ba microservice

5.1 Trip Service

Trách nhiệm: khởi tạo chuyến đi, giữ và chuyển trạng thái (State pattern), phát sự kiện domain. **API tiêu biểu:** `POST /trips`, `POST /trips/{id}/driver-accept`, `POST /trips/{id}/pickup`, `POST /trips/{id}/dropoff`.

5.2 Matching và Pricing Service

Trách nhiệm: tìm tài xế gần nhất (geo-query trên MongoDB), tính giá theo *Strategy pattern* (giá thường / surge / khuyến mãi). **API:** `POST /match`, `POST /quote`.

5.3 Payment Service

Trách nhiệm: tạo giao dịch thanh toán, ghi sổ cái, xử lý hoàn tiền và *bồi thường* khi một Saga thất bại. Service phải hỗ trợ idempotency để cùng một yêu cầu không bị trừ tiền hai lần. Trong hệ thống thật, Payment Service chỉ lưu token do cổng thanh toán cấp, không lưu số thẻ/CVV. **API:** `POST /payments`, `POST /payments/{id}/refund`.

6 Áp dụng nguyên tắc SOLID (kèm ví dụ .NET)

6.1 S – Single Responsibility Principle

Mỗi lớp chỉ có *một lý do để thay đổi*. Tách tính giá khỏi lưu trữ.

```
// SAI: lam ca tinh gia LAN luu DB trong mot lop
// DUNG: tach ra
public sealed class FareCalculator           // chi tinh gia
{
    public decimal Calculate(Trip trip, IPricingStrategy strategy)
        => strategy.GetPrice(trip);
}

public sealed class TripRepository            // chi truy xuat du lieu
{
    private readonly TripDbContext _db;
    public TripRepository(TripDbContext db) => _db = db;
    public async Task SaveAsync(Trip trip)
    {
        _db.Trips.Update(trip); // vi du rut gon: them hoac cap nhat entity
        await _db.SaveChangesAsync();
    }
}
```

Listing 1: SRP: mỗi lớp một trách nhiệm

6.2 O – Open/Closed Principle

Mở để mở rộng, đóng để sửa đổi. Thêm chiến lược giá mới mà không sửa code cũ.

```
public interface IPricingStrategy
{
    decimal GetPrice(Trip trip);
}

public sealed class StandardPricing : IPricingStrategy
{
    public decimal GetPrice(Trip trip)
        => trip.Distance * 1.0m + trip.EstimatedTime * 0.5m;
```

```
}

// Mo rong them ma KHONG sua StandardPricing
public sealed class SurgePricing : IPricingStrategy
{
    private readonly IPricingStrategy _basePricing;
    private readonly decimal _multiplier;
    public SurgePricing(
        IPricingStrategy basePricing, decimal multiplier)
        => (_basePricing, _multiplier) = (basePricing, multiplier);

    public decimal GetPrice(Trip trip)
        => _basePricing.GetPrice(trip) * _multiplier;
}
```

Listing 2: OCP: thêm SurgePricing không đụng đến lớp cũ

6.3 L – Liskov Substitution Principle

Lớp triển khai phải thay thế được abstraction mà không phá vỡ hành vi mong đợi. Trong ví dụ, "giá không âm" là hậu điều kiện cần được ghi rõ và kiểm thử; phép kiểm tra runtime chỉ là hàng rào bảo vệ, không tự nó chứng minh LSP.

```
public decimal Quote(Trip trip, IPricingStrategy strategy)
{
    decimal price = strategy.GetPrice(trip);
    // Bat ky chien luoc nao cung phai tuan thu hop dong: gia khong am
    if (price < 0) throw new InvalidOperationException("Gia khong hop le");
    return price;
}
```

Listing 3: LSP: mọi IPricingStrategy đều trả về giá hợp lệ (không âm)

6.4 I – Interface Segregation Principle

Tách interface nhỏ thay vì một interface "béo".

```
public interface ITripReader
{
    Task<Trip?> GetByIdAsync(Guid id);
}
public interface ITripWriter
{
    Task SaveAsync(Trip trip);
}
// Service chi doc thi chi phu thuoc ITripReader -> it coupling hon
```

Listing 4: ISP: tách quyền đọc và ghi

6.5 D – Dependency Inversion Principle

Module cấp cao phụ thuộc *trừu tượng*, không phụ thuộc chi tiết. Dùng Dependency Injection sẵn có của .NET.

```
public sealed class TripService
{
    private readonly ITripWriter _writer;   // truu tuong, khong phai SQL cu the
    private readonly IEventBus _bus;
    public TripService(ITripWriter writer, IEventBus bus)
        => (_writer, _bus) = (writer, bus);

```

```
    public async Task AcceptAsync(Trip trip)
    {
        trip.Accept();
        await _writer.SaveAsync(trip);
        await _bus.PublishAsync(new TripAccepted(trip.Id));
    }
}

// Program.cs - dang ky container DI
builder.Services.AddScoped<ITripWriter, TripRepository>();
builder.Services.AddSingleton<IEventBus, RabbitMqEventBus>();
```

Listing 5: DIP: tiêm phụ thuộc qua interface

7 Các design pattern sử dụng trong SwiftRide

Pattern	Loại	Ứng dụng trong SwiftRide
State	Behavioral	Vòng đời chuyến đi và các transition hợp lệ.
Strategy	Behavioral	Chiến lược tính giá (Standard / Surge / Promo).
Repository	(DDD)	Trừu tượng hóa truy cập CSDL cho mỗi service.
Factory	Creational	Tạo đối tượng `PaymentMethod` (Card/Wallet).
Observer/Pub-Sub	Behavioral	Phát sự kiện domain qua RabbitMQ.
API Gateway	Architectural	Một điểm vào cho client (dùng YARP).
Saga	Architectural	Quy trình nhiều service và hành động bồi thường.
Outbox	Architectural	Ghi dữ liệu và event tin cậy trong cùng transaction.

```
// Vi du rut gon de minh hoa pattern. Bang trang thai nghiep vu day du:
// Requested -> DriverAccepted -> EnRoute -> PickedUp -> DroppedOff
// -> PaymentPending -> Paid; nhanh ket thuc: Cancelled / Failed.
public abstract class TripState
{
    public abstract TripState Accept();
    public abstract TripState Complete();
}

public sealed class RequestedState : TripState
{
    public override TripState Accept()   => new AcceptedState();
    public override TripState Complete() =>
        throw new InvalidOperationException("Chua duoc chap nhan");
}
public sealed class AcceptedState : TripState
{
    public override TripState Accept()   =>
        throw new InvalidOperationException("Da chap nhan roi");
    public override TripState Complete() => new CompletedState();
}
```

```
public sealed class CompletedState : TripState
{
    public override TripState Accept()   => this;
    public override TripState Complete() => this;
}
```

Listing 6: State pattern cho trạng thái chuyến đi

8 Ngăn xếp công nghệ .NET

Thành phần	Công nghệ đề xuất
Runtime / Framework	.NET 10 (LTS), ASP.NET Core Minimal API / Controllers
ORM / Data	EF Core 10 (SQL Server, PostgreSQL), MongoDB.Driver
API Gateway	**YARP** (phù hợp với hệ sinh thái .NET)
Message Broker	RabbitMQ (qua MassTransit)
Xác thực	OAuth 2.0 / OpenID Connect, JWT Bearer
Container	Docker + docker-compose
Kiểm thử	xUnit + Moq
Quan sát hệ thống	OpenTelemetry (log, metric, distributed trace)

8.1 Cấu trúc thư mục đề xuất

```
SwiftRide/
 |- src/
 |    |- ApiGateway/                 # YARP, dinh tuyen + JWT
 |    |- TripService/
 |    |    |- Domain/                # Trip, TripState (State pattern)
 |    |    |- Application/           # TripService, DTOs
 |    |    |- Infrastructure/        # EF Core + SQL Server
 |    |    |- Api/                   # Controllers / Minimal API
 |    |- MatchingService/
 |    |    |- Domain/ Application/ Infrastructure(MongoDB)/ Api/
 |    |- PaymentService/
 |         |- Domain/ Application/ Infrastructure(PostgreSQL)/ Api/
 |- tests/
 |    |- TripService.Tests/          # xUnit + Moq
 |- docker-compose.yml               # 3 service + 3 CSDL + RabbitMQ
 |- SwiftRide.sln
```

Listing 7: Solution layout cho SwiftRide

9 Trade-off và yêu cầu chất lượng

9.1 Kiến trúc mã nguồn

Microservices không mặc định tốt hơn monolith. Với nhóm nhỏ, nghiệp vụ chưa ổn định và tải thấp, **modular monolith** thường dễ phát triển, kiểm thử và triển khai hơn. Trong bài tập lớn dùng kiến trúc microservices khi cần triển khai/mở rộng độc lập, ranh giới nghiệp vụ đủ rõ và chấp nhận chi phí của hệ phân tán.
Trong SwiftRide, microservices là lựa chọn phục vụ học tập. Một lộ trình hợp lý là: **(1)** xác định module và bounded context; **(2)** chạy luồng cơ bản; **(3)** tách service và database; **(4)** thêm messaging, Outbox và observability.

9.2 Nhất quán và độ tin cậy

- **Strong consistency** bên trong một service; **eventual consistency** giữa các service.
- **Transactional Outbox** bảo đảm thay đổi dữ liệu và ghi event vào Outbox trong cùng transaction; worker sẽ publish event sau đó.
- Consumer phải **idempotent**, vì broker thường cung cấp giao nhận *at least once*; cùng một message có thể đến nhiều lần.
- Mọi lời gọi mạng cần timeout. Chỉ retry lỗi tạm thời, dùng exponential backoff + jitter; message lỗi nhiều lần được chuyển vào **dead-letter queue**.
- Saga điều phối/choreography nhiều bước nghiệp vụ và định nghĩa hành động bồi thường. Saga không thay thế transaction nội bộ hay Outbox.

9.3 Bảo mật và khả năng quan sát

- Dùng OAuth 2.0/OpenID Connect; Gateway xác thực, service vẫn phải kiểm tra quyền truy cập đối với tài nguyên.
- Không log token, số thẻ hoặc dữ liệu cá nhân nhạy cảm; secret không đặt trực tiếp trong source code.
- Mỗi request/message mang `CorrelationId`/trace context. Dùng OpenTelemetry để liên kết log, metric và distributed trace.
- Theo dõi các chỉ số dễ hiểu: tỉ lệ đặt xe thành công, thời gian matching, độ trễ API, tỉ lệ thanh toán lỗi và độ dài hàng đợi.